

PRÁCTICA 1

Conversión a VHDL



UNIVERSIDAD DE MÁLAGA



de

Raúl Escudero Jiménez

José Luis Pérez Martín

para la asignatura de

ELECTRÓNICA DIGITAL

Tutorizado por

Javier López García

Versiones del Módulo Cuenta4_votos en lenguaje VHDL	5
Versión ecuaciones lógicas	6
Versión sentencias IF	7
Comentario sobre la versión sentencia if:	8
Versión Case	8
SIMULACIÓN MEDIANTE TEST BENCH	9
Resultados de Simulación	12
1. Método ecuaciones lógicas	12
2. Método IF	12
3. Método Case	12

Versiones del Módulo Cuenta4_votos en lenguaje VHDL

En esta sección se muestra la misma lógica empleada en los *schematics* en lenguaje VHDL. Para ello empleamos tres versiones distintas, desarrolladas a partir de la tabla de verdad y su posterior simplificación mediante Karnaugh (disponible en el primer documento de la práctica 1).

```
--
--
-- Company:
-- Engineer:
--
-- Create Date:      10:11:30 04/16/2026
-- Design Name:
-- Module Name:      Cuenta_4votos - Behavioral
-- Project Name:
-- Target Devices:
-- Tool versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
--
```

```
--
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if
instantiating
-- any Xilinx primitives in this code.
```

```

--library UNISIM;
--use UNISIM.VComponents.all;

entity Cuenta_4votosVHDL is
    Port ( M : in  STD_LOGIC_VECTOR (3 downto 0);
          S : out  STD_LOGIC_VECTOR (2 downto 0));
end Cuenta_4votosVHDL;

architecture Behavioral of Cuenta_4votosVHDL is

begin -- Modulo cuenta 4 votos
    process (M)
    begin

```

Versión ecuaciones lógicas

```

-- VERSION ECUACION LOGICA
    --S(0) <= M(0) XOR M(1) XOR M(2) XOR M(3);

    --S(1) <= ((NOT(M(3))) AND M(2) AND M(0)) OR
        --      ((NOT(M(2))) AND M(0) AND M(1)) OR
        --      ((NOT(M(0))) AND M(1) AND M(2)) OR
        --      ((NOT(M(2))) AND M(3) AND M(1)) OR
        --      ((NOT(M(3))) AND M(0) AND M(2)) OR
        --      ((NOT(M(1))) AND M(3) AND M(2)) OR
        --      ((NOT(M(1))) AND M(3) AND M(0));

    --      S(2) <= M(3) AND M(2) AND M(1) AND M(0);

```

Versión sentencias IF

```
-- VERSION IF
      --if (M(0) XOR M(1) XOR M(2) XOR M(3)) = '1'
then -- Igualar a uno para devolver valor booleano
      -- S(0) <= '1';
      --else
      -- S(0) <= '0';
      --end if;

      --if (((NOT(M(3))) AND M(2) AND M(0)) OR
      -- ((NOT(M(2))) AND M(0) AND M(1)) OR
      -- ((NOT(M(0))) AND M(1) AND M(2)) OR
      -- ((NOT(M(2))) AND M(3) AND M(1)) OR
      -- ((NOT(M(3))) AND M(0) AND M(2)) OR
      -- ((NOT(M(1))) AND M(3) AND M(2)) OR
      -- ((NOT(M(1))) AND M(3) AND M(0))) = '1'
then

      -- S(1) <= '1';
      --else
      -- S(1) <= '0';
      --end if;

      --if (M(0) AND M(1) AND M(2) AND M(3)) = '1'
then

      -- S(2) <= '1';
      --else
      -- S(2) <= '0';
      --end if;
```

Comentario sobre la versión sentencia if:

Dado que la sentencia if requiere una condición de tipo *boolean*, es necesario comparar la expresión lógica $(M(0) \text{ XOR } M(1) \text{ XOR } M(2) \text{ XOR } M(3))$ —cuyo resultado es de tipo *std_logic*, 1 ó 0— con el valor '1' para obtener un resultado booleano que pueda usarse en el if.

En otras palabras, cuando la expresión lógica valga '1' la comparación devolverá el valor booleano TRUE y se ejecutará la asignación de la salida. En caso contrario, devolverá FALSE y saltará a “else”.

Versión Case

```
-- Version Case
      case M(3 downto 0) is
        when "0000" => S <= "000";
        when "0001" => S <= "001";
        when "0010" => S <= "001";
        when "0011" => S <= "010";
        when "0100" => S <= "001";
        when "0101" => S <= "010";
        when "0110" => S <= "010";
        when "0111" => S <= "011";
        when "1000" => S <= "001";
        when "1001" => S <= "010";
        when "1010" => S <= "010";
        when "1011" => S <= "011";
        when "1100" => S <= "010";
        when "1101" => S <= "011";
        when "1110" => S <= "011";
        when "1111" => S <= "100";
        when others => S <= "000";
      end case;

      end process;
end Behavioral;
```

SIMULACIÓN MEDIANTE TEST BENCH

En esta sección se muestra el archivo test bench compartido por las tres versiones VHDL. Además, se adjuntan los resultados de la simulación.

```
-----
-- Company:
-- Engineer:
--
-- Create Date:    12:13:43 04/16/2026
-- Design Name:
--
--                                Module                               Name:
C:/Users/Usuario_UMA/Downloads/RLab_Plant_Jose_Raul_16-04-26/Cuenta_4votosVHDL_Test.vhd
-- Project Name:  RLab_Plant
-- Target Device:
-- Tool versions:
-- Description:
--
--   VHDL   Test   Bench   Created   by   ISE   for   module:
Cuenta_4votosVHDL
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-- Notes:
--   This testbench has been automatically generated using
types std_logic and
--   std_logic_vector for the ports of the unit under test.
Xilinx recommends
--   that these types always be used for the top-level I/O of a
design in order
--   to guarantee that the testbench will bind correctly to the
post-implementation
```

```

-- simulation model.
-----

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;

ENTITY Cuenta_4votosVHDL_Test IS
END Cuenta_4votosVHDL_Test;

ARCHITECTURE behavior OF Cuenta_4votosVHDL_Test IS

    constant CkSemiPeriod : TIME := 50 ns;

    COMPONENT Cuenta_4votosVHDL
    PORT(
        M : IN  std_logic_vector(3 downto 0);
        S : OUT std_logic_vector(2 downto 0)
    );
    END COMPONENT;

    signal M : std_logic_vector(3 downto 0) := (others =>
'0');

    signal S : std_logic_vector(2 downto 0);

BEGIN

    UUT: Cuenta_4votosVHDL
        port map (
            M => M,
            S => S
        );

    clock1: process
begin

```

```

        M(0) <= '0';
    loop
        wait for CKSemiPeriod;
        M(0) <= not M(0);
    end loop;
end process;

clock2: process
begin
    M(1) <= '0';
    loop
        wait for 2*CKSemiPeriod;
        M(1) <= not M(1);
    end loop;
end process;

clock3: process
begin
    M(2) <= '0';
    loop
        wait for 4*CKSemiPeriod;
        M(2) <= not M(2);
    end loop;
end process;

clock4: process
begin
    M(3) <= '0';
    loop
        wait for 8*CKSemiPeriod;
        M(3) <= not M(3);
    end loop;
end process;

END behavior;

```

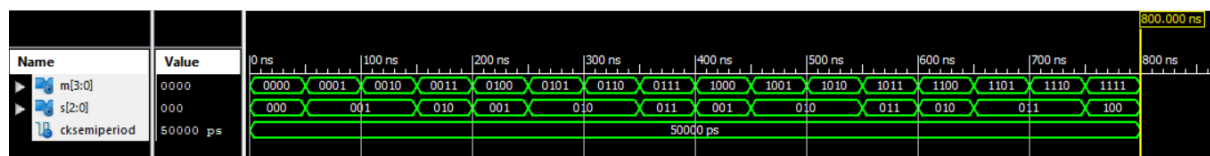
Nota sobre la creación de un nuevo archivo Test Bench

Se detectó que Xilinx ISE seguía compilando el archivo Cuenta_4_TEST, aunque no estuviera ya incluido al proyecto y el usuario estuviera simulando Cuenta_4votosVHDL_Test. Esto ocurrió porque ISE mantiene referencias estáticas a los ficheros añadidos al proyecto y no actualiza automáticamente los vínculos cuando existen copias en distintas rutas. Para corregirlo, fue necesario eliminar ambos archivos del proyecto, borrar Cuenta_4_TEST y volver a añadir manualmente Cuenta_4votosVHDL_Test, tras lo cual la compilación y simulación se realizaron correctamente.

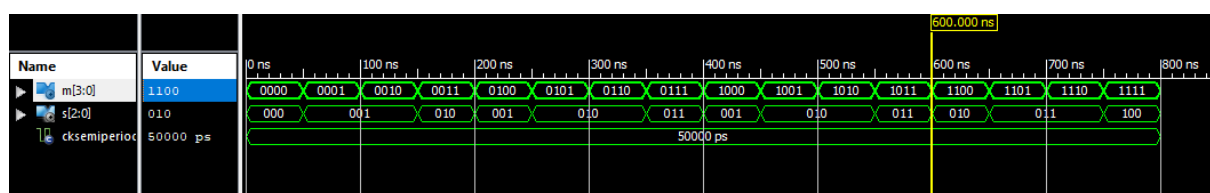
Resultados de Simulación

A continuación se muestran los resultados de simulación. Se comprueba que son equivalentes y el módulo VHDL cuenta correctamente los votos a favor.

1. Método ecuaciones lógicas



2. Método IF



3. Método Case

